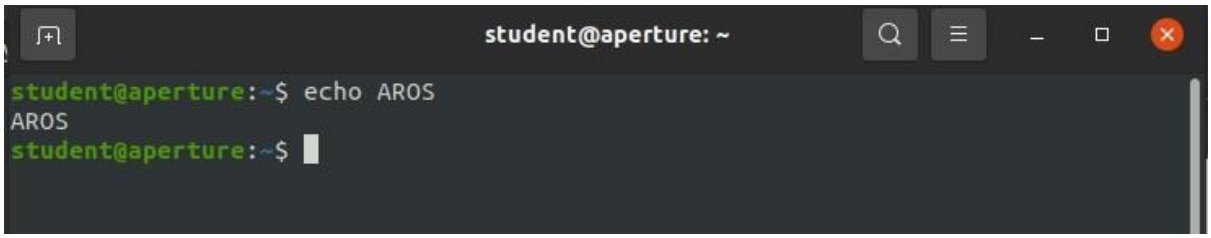
Arhitektura računara i operativni sistemi

Laboratorijske vežbe 3

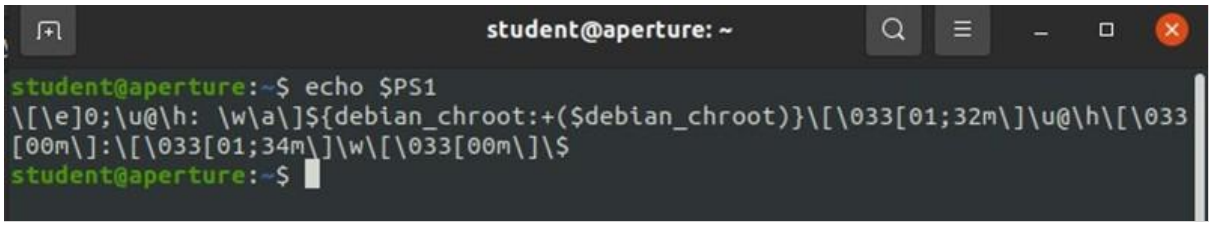
1. Naredba echo

Kada naredbi *echo* prosledimo neki argument, ona će ga ispisati na standardni izlaz (terminal):



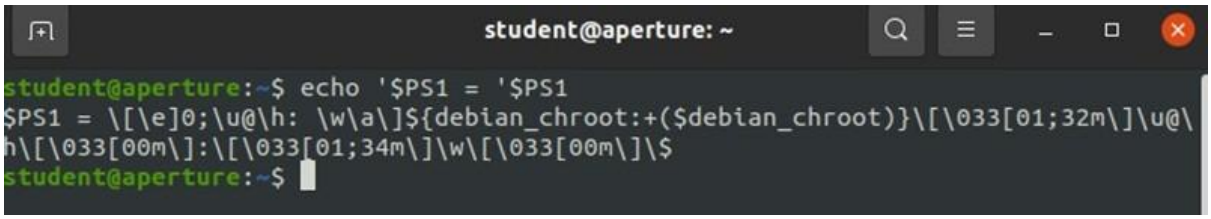
Slika 1 - Naredba echo

Ista naredba nam može ispisati vrednosti različitih promenljivih, recimo, promenljive PS1 u kojoj se nalaze podešavanja *bash prompt*-a. Da bi se ispisala vrednost neke promenljive, ispred njenog naziva stavljamo znak $.



Slika 2 - Ispis vrednosti promenljive

Obratite pažnju na sledeće:

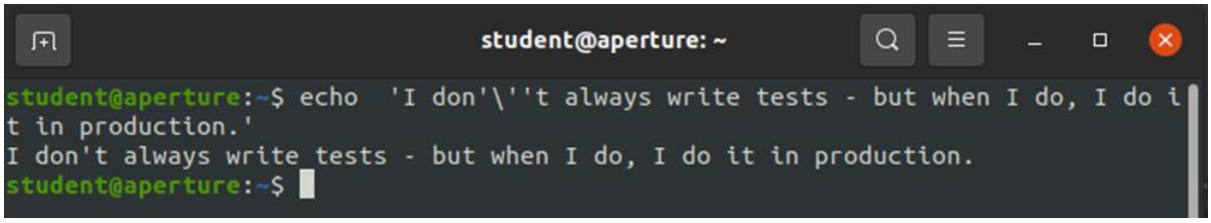


Slika 3 - Upotreba jednostrukih navodnika

Primetite da je jedan deo stringa koji se ispisuje stavljen pod jednostruke navodnike. Ovi navodnici omogućavaju da se u terminalu ispiše tačno ta vrednost koja se nalazi između navodnika - literal. Dakle, $PS1 koje se nalazi između navodnika se neće zameniti vrednošću ove promenljive, što i vidimo u rezultatu ispisa. Drugo po redu $PS1 se ne nalazi između

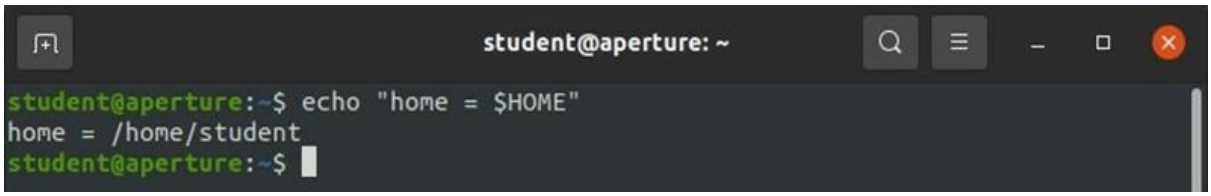
jednostrukih navodnika, pa će se u terminalu ispisati vrednost te promenljive, a to je konfiguracija *bash prompt*-a.

Ukoliko želimo da ispišemo rečenicu „I don't always write tests - but when I do, I do it in production", koja **sadrži u sebi znak '**, moramo iskoristiti znak „\" (*backslash* tj. *escape karakter*) i to na sledeći način:



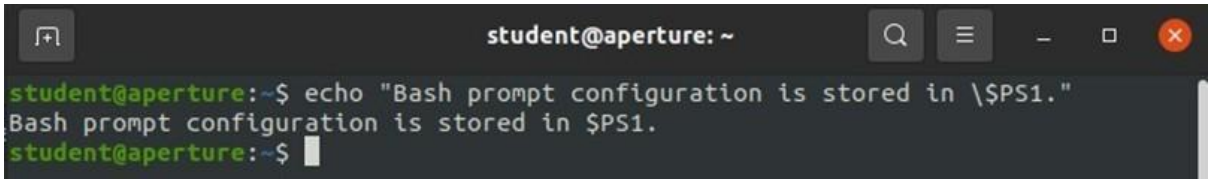
Slika 4 - Upotreba escape karaktera

Ukoliko se promenljiva nađe između dvostrukih navodnika (") biće zamenjena svojom vrednošću.



Slika 5 - Upotreba dvostrukih navodnika

Ukoliko unutar dvostrukih navodnika ne želimo da se promenljiva zameni svojom vrednošću, to ćemo takođe izbeći primenom *escape* karaktera, odnosno, znaka „\". Na primer:



Slika 6 - Upotreba escape karaktera

2. Bash skriptovi

Sve naredbe koje smo do sada pokretali u terminalu mogu se zapisati i u tekstualnim fajlovima zvanim „skriptovi", pa se čitav niz naredbi može izvršiti pokretanjem samo jednog fajla. Pisanje skriptova se u praksi veoma često koristi kada je potrebno automatizovati operacije koje se svakodnevno obavljaju. Skriptovi treba da budu mali i jednostavni, a ako posao krene da se komplikuje, treba da odaberete neki drugi skriptni jezik.

Skript započinjmo sledećim zaglavljem: `#!/bin/bash` poznatim kao *šebeng* (engl. *Shebang*).

2.1. Primer 1 – promenljiva, komentar, naredba

Napisaćemo *bash* skript koji ispisuje neku poruku i trenutni datum i vreme:

```
student@aperture: ~
  GNU nano 4.8                    welcome.sh
#!/bin/bash

#Ovo je komentar.

now=$(date);
echo "Dobro dosli! Datum i vreme: $now";
```

Slika 7 - Skript koji ispisuje poruku, datum i vreme

U *bash*-u, znak # označava komentar. U ovom skriptu možemo uočiti dva komentara. Kreirana je jedna promenljiva sa nazivom *now*. Toj promenljivoj smo dodelili vrednost koju vraća naredba *date*, a to je trenutni datum i vreme. U sledećoj liniji, pomoću naredbe *echo*, ispisujemo poruku i vrednost promenljive *now*. Obratite pažnju da ispred promenljive *now* moramo staviti znak $ kako bi se reč *now* protumačila kao promenljiva, te da bi se ispisala njena vrednost. Na sledećoj slici je prikazan rezultat izvršavanja ovog skripta, koji smo nazvali *welcome.sh*:

```
student@aperture: ~
student@aperture:~$ nano welcome.sh
student@aperture:~$ ./welcome.sh
Dobro dosli! Datum i vreme: уто, 29. нов 2022.  18:42:50 CET
student@aperture:~$
```

Slika 8 - Pokretanje skripta

Na prethodnoj slici vidimo da se skript pokreće tako što u terminalu unesemo znakove ./ i naziv skripta. Ukoliko se skript nalazi na nekoj drugoj lokaciji, navešćemo putanju do tog skripta.

2.2. Primer 2 – čitanje fajla

Prethodni primer ćemo nadograditi tako da za današnji datum pročita i ispiše sve zadatke koje smo zapisali sebi u fajlu *notes.txt*. U fajlu su zadaci zapisani na sledeći način:

```
student@aperture: ~
  GNU nano 4.8                    notes.txt
21.11.2022. Uciti AROS
22.11.2022. Kolokvijum iz AROS-a
29.11.2022. Ocistiti stan
29.11.2022. Instalirati Fedora VM
30.11.2022. Vezbati BASH
30.11.2022. Uraditi domaci iz AROS-a
10.12.2022. Ukrasiti jelku
```

Slika 9 - Sadržaj fajla notes.txt

Dakle, po jedan zadatak se nalazi u svakom redu, a prethodi mu datum u formatu dd.MM.yyyy. Da bismo pronašli sve zadatke koje moramo danas obaviti, potrebno je da filtriramo sadržaj fajla *notes.txt*, tako da nam se prikažu samo oni redovi koji sadrže današnji datum. To možemo postići korišćenjem već dobro poznatog programa ***grep***. Obratite pažnju da je datum formatiran, pa ne možemo tražiti redove u kojima se nalazi ceo izlaz naredbe *date*, već on mora biti u formatu dd.MM.yyyy. Izlaz naredbe *date* se može formatirati na sledeći način:

```
date +"%d.%m.%Y."
```

```
student@aperture: ~
  GNU nano 4.8                    welcome.sh
#!/bin/bash

#Ovo je komentar.

now=$(date);
echo "Dobro dosli! Datum i vreme: $now";
echo "Zadaci:";
grep $(date +"%d.%m.%Y.") notes.txt;
```

Slika 10 - Filtriranje sadržaja fajla

Naš skript smo dopunili linijom koja će, pomoću naredbe *grep,* iz fajla *notes.txt* izdvojiti one linije koje u sebi sadrže današnji datum u formatu dd.MM.yyyy. Kada pokrenemo skript, dobijemo rezultat:

```
student@aperture: ~
student@aperture:~$ ./welcome.sh
Dobro dosli! Datum i vreme: уто, 29. нов 2022.  22:11:57 CET
Zadaci:
29.11.2022. Ocistiti stan
29.11.2022. Instalirati Fedora VM
student@aperture:~$
```

Slika 11 - Rezultat izvršavanja skripta

2.3. Primer 3 – argumenti

Do sada smo videli da se naredbe u terminalu mogu pokrenuti sa različitim opcijama, a možemo im prosleđivati i argumente. Isto možemo uraditi i sa našim skriptom. Promenljive čiji su nazivi prirodni brojevi ($1, $2, $3 ...) predstavljaju, redom, opcije i argumente koje smo prosledili prilikom pokretanja skripta. Posmatrajmo sledeći skript:

```
student@aperture: ~
GNU nano 4.8                    script.sh
#!/bin/bash

echo $1;
echo $2;
echo $3;
```

Slika 12 - Skript za ispis argumenata

Skript na slici iznad ispisuje prva tri argumenta koja mu se proslede. Pogledajmo rezultate izvršavanja ovog skripta:

```
student@aperture: ~
student@aperture:~$ ./script.sh



student@aperture:~$ ./script.sh debian fedora arch
debian
fedora
arch
student@aperture:~$ ./script.sh debian fedora
debian
fedora
```

Slika 13 - Rezultat izvršavanja skripta sa argumentima

U prvom slučaju smo pokrenuli skript bez argumenata, pa se u terminalu tri puta ispisuje prazan red. Zatim je skript pokrenut sa tri argumenta (*debian fedora arch*). U ovom slučaju promenljiva $1 uzima vrednost *debian*, promenljiva $2 vrednost *fedora*, a promenljiva $3 *arch*. Naposletku je skript pokrenut sa dva argumenta, pa se u terminalu ispisuju tva argumenta i prazan red. Promenljiva $0 sadrži ime skripta.

Naš skript *welcome.sh* ćemo unaprediti dodavanjem opcije -a koja će nam omogućiti da dodamo novi zadatak u fajl *notes.txt*. Naredba bi trebalo da izgleda ovako:

```
./welcome.sh -a 11.12.2022. 'Vezbati programiranje 2'
```

Dakle, nakon opcije -a, unosimo datum kao drugi argument, a tekst zadatka kao treći.

```
student@aperture: ~
  GNU nano 4.8                    welcome.sh
#!/bin/bash

#Ovo je komentar.

if [ "$1" = "-a" ];
then
       echo "$2 $3" >> notes.txt;
fi

now=$(date);
echo "Dobro dosli! Datum i vreme: $now";
echo "Zadaci:";
grep $(date +"%d.%m.%Y.") notes.txt;
```

Slika 14 – if

Potrebno je pre svega da proverimo vrednost prvog prosleđenog argumenta, kako bismo znali šta je potrebno izvršiti u zavisnosti od njegove vrednosti. Ukoliko naredba `[ "$1" = "-a" ]` vraća izlazni kod 0 (iskaz unutar uglastih zagrada je tačan), izvršiće se naredbe koje se nalaze nakon ključne reči *then*. Kraj ovog bloka označen je ključnom rečju *fi*. Naravno, postoji i blok *else*, ali ga nismo koristili u ovom slučaju. Ukoliko je uslov ispunjen, vrednosti drugog i trećeg argumenta se upisuju u fajl *notes.txt*. Za upis smo koristili znak >> kojim izlaz naredbe *echo* preusmeravamo u fajl *notes.txt* i to tako da se novi zadatak **doda nakon niza postojećih zadataka (*append*)**. Primetite da smo do sada za preusmeravanje izlaza naredbe koristili znak > (samo jedan). Razlika između > i >> je u tome što će znak > postaviti sadržaj fajla na novu vrednost (ono što je prethodno pisalo u fajlu će biti obrisano), dok će znak >> dodati novi sadržaj na ono što u fajlu već postoji.

```
student@aperture: ~
student@aperture:~$ ./welcome.sh -a 11.12.2022. 'Vezbati programiranje 2'
Dobro dosli! Datum i vreme: уто, 29. нов 2022.  23:38:29 CET
Zadaci:
29.11.2022. Ocistiti stan
29.11.2022. Instalirati Fedora VM
student@aperture:~$ ./welcome.sh -a 29.11.2022. 'Uciti Javu'
Dobro dosli! Datum i vreme: уто, 29. нов 2022.  23:39:38 CET
Zadaci:
29.11.2022. Ocistiti stan
29.11.2022. Instalirati Fedora VM
29.11.2022. Uciti Javu
student@aperture:~$
```

Slika 15 - Izvršavanje skripta sa argumentima

2.4. Primer 4 – logičko i, ili, aritmetička poređenja, for petlja

Sa logičkim „i" i logičkim „ili" ste se do sada već susreli. Posmatrajmo sledeći izraz:

```
if [ $1 = arch ] && [ $2 = fedora ];
```

Ukoliko prva naredba `[ $1 = arch ]` ne vrati izlazni kod 0 (jednakost nije tačna), naredba *if* će koristiti taj izlazni kod (ceo izraz je netačan, uslov nije ispunjen). Ukoliko prva naredba `[ $1 = arch ]` vrati izlazni kod 0, odnosno, jednakost je tačna, proverava se izlazni kod druge naredbe `[ $2 = fedora ]`, te će ga *if* naredba koristiti.

Kod logičkog „ili" se koristi izlazni kod prve naredbe ukoliko je 0, odnosno, izlazni kod druge naredbe, ako prvi nije 0.

Za razliku od znakovnih nizova (stringova) koji se porede znakom =, u *bash*-u se brojevi porede operatorima:

-eq ispituje da li je prvi argument jednak drugom

-ne ispituje da li je prvi argument različit od drugog

-lt ispituje da li je prvi argument manji od drugog

-gt ispituje da li je prvi argument veći od drugog

-le ispituje da li je prvi argument jednak ili manji od drugog

-ge ispituje da li je prvi argument jednak ili veći od drugog

Posmatrajmo sledeći primer:

Slika 16 - for petlja

Vrednost promenljive *i* će se ispisati ukoliko je manja od prvog argumenta koji prosledimo skriptu **ili** veća od drugog. Uslov se proverava za vrednosti od 1 do 20 u *for* petlji.

```
student@aperture: ~
student@aperture:~$ ./operators.sh 4 12
1
2
3
13
14
15
16
17
18
19
20
student@aperture:~$
```

Slika 17 - Izvršavanje skripta

2.5. Primer 5 – case, funkcije

Ukoliko ćemo neku sekvencu koda koristiti više puta, najbolje je da je odvojimo u zasebnu funkciju, kao što smo do sada činili i u drugim jezicima.

```
student@aperture: ~
  GNU nano 4.8                    functions.sh
#!/bin/bash

respond(){
        case $1 in
                hi)
                        echo 'Hello!'
                ;;
                bye)
                        echo 'Good bye!'
                ;;
                howRU)
                        echo 'Fine, how are you?'
                ;;
                whoRU)
                        whoami
                ;;
                *)
                        echo "I don't know what to say"
                ;;
        esac
}

respond $1;
respond $2;
```

Slika 18 - Funkcija i case

U skriptu *functions.sh* napisali smo funkciju *respond*. Funkciju pozivamo samo navodeći njen naziv. Iako u potpisu funkcije nema parametara, pri pozivu joj možemo proslediti argumente. U ovom slučaju smo pri oba poziva prosledili po jedan argument, i to prvi i drugi argument koji su prosleđeni samom skriptu. Unutar funkcije, prosleđeni argument takođe referenciramo kao promenljivu čiji je naziv prirodan broj. Pošto smo prosledili jedan argument, njegovu

vresnost ćemo dobiti koristeći promenljivu $1. Dakle, u funkciji *respond* se $1 ne odnosi na prvi argument prosleđen skriptu, **već na prvi argument prosleđen toj funkciji**. Unutar funkcije *respond* se vrednost prosleđenog argumenta ispituje pomoću *case* bloka.

```
student@aperture: ~
student@aperture:~$ ./functions.sh hi whoRU
Hello!
student
student@aperture:~$ ./functions.sh hi howRU
Hello!
Fine, how are you?
student@aperture:~$
```

Slika 19 - Izvršavanje skripta

3. Alijasi

Postoji dosta komandi koje često pokrećemo, a čiji zapis nije naročito kratak. Linuks nam nudi mogućnost da uštedimo vreme kucanja komandi, tako što ćemo im dati alternativni naziv, takozvani alijas (engl. *alias*). Za rad sa alijasima koristi se komanda *alias*. Ukoliko ovoj komandi ne prosledimo nikakav parametar, vraća nam spisak svih alijasa za trenutnog korisnika:

```
student@aperture: ~
student@aperture:~$ alias
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo terminal || echo
 error)" "$(history|tail -n1|sed -e '\''s/^\s*[0-9]\+\s*//;s/[;&|]\s*alert$//'\'
')"'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -alF'
alias ls='ls --color=auto'
```

Slika 20 - Komanda alias

Na slici vidimo više alijasa. Jedan od njih je **`la`**. Ako pokrenemo naredbu **`la`**, ono što će se zapravo izvršiti je **`ls -A`**.

Za kreiranje novog alijasa komanda je sledeća:

```
alias naziv_alijasa=”komanda”
```

Kao naziv alijasa, unosimo naziv kojim ćemo moći da referenciramo komandu navedenu desno od znaka jednakosti. Na primer, za skript *welcome.sh* koji smo napravili u prethodnoj sekciji, možemo napraviti dva alijasa:

```
alias notes=”/home/student/welcome.sh”
```

```
alias newnote=”/home/student/welcome.sh -a”
```

Sada je moguće pokrenuti skript *welcome.sh* upisujući naziv odgovarajućeg alijasa u terminal.

```
student@aperture:~$ newnote 02.12.2022. "Pregledati kolokvijume iz AROS-a"
Dobro dosli! Datum i vreme: пет,  2. дец 2022.  19:11:53 CET
Zadaci:
02.12.2022. Pregledati kolokvijume iz AROS-a
```

Slika 21 - Izvršavanje skripte pomoću alijasa

Na ovaj način kreiraju se privremeni alijasi. Ukoliko zatvorimo terminal i ponovo ga otvorimo, videćemo da su alijasi koje smo kreirali nestali. Za kreiranje trajnih alijasa potrebno je da gorenavedene komande unesemo u fajl *~/.bashrc*, koji predstavlja konfiguracioni fajl *bash shell*-a.

```
student@aperture: ~
GNU nano 4.8                    .bashrc
    alias ls='ls --color=auto'
    #alias dir='dir --color=auto'
    #alias vdir='vdir --color=auto'

    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

# colored GCC warnings and errors
#export GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=01:q>

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'
alias notes='/home/student/welcome.sh'
alias newnote='/home/student/welcome.sh -a'
```

Slika 22 - Podešavanje alijasa u fajlu .bashrc

Napomena: u skriptu *welcome.sh* smo izmenili putanju do fajla notes.txt na ~/notes.txt .

Ukoliko želimo da obrišemo neki alijas, koristimo sledeću komandu:

```
unalias naziv_alijasa
```

4. Cron

Cron je program za zakazivanje poslova (eng. *job scheduler*) na UNIX operativnim sistemima. Poslovi koji se zakazuju pomoću *cron*-a nazivaju se *cron* poslovi (engl. *cron jobs*). *Cron* je *daemon* – pozadinski proces koji izvršava neinteraktivne poslove. *Cron* fajl je tekstualni fajl koji sadrži komande koje će biti pokretane u određeno vreme, periodično. Podrazumevana lokacija ovog fajla je */etc/crontab*.

Za kreiranje i izmenu *cron* fajlova koristi se komanda:

```
crontab -e
```

Ukoliko fajl ne postoji na sistemu, gorenavedena komanda će ga napraviti. Da bi se videli svi zakazani poslovi, komandi se prosleđuje opcija `-l`:

```
crontab -l
```

Za brisanje svih zakazanih poslova koristi se komanda:

```
crontab -r
```

4.1. Crontab sintaksa

Linija u *cron* fajlu se sastoji od pet polja sa sledećim vrednostima:

- **Minuti** – u koliko minuta će se komanda pokretati, u opsegu od 0 do 59
- **Sati** – u koliko sati će se komanda pokretati, u opsegu od 0 do 23
- **Dan u mesecu** – dan u mesecu u kojem će se komanda pokretati, u opsegu od 1 do 31
- **Mesec** – mesec u kojem će se komanda pokretati, u opsegu od 1 do 12 (od januara do decembra)
- **Dan u nedelji** – dan u nedelji u kojem će se komanda pokretati, u opsegu od 0 do 6 (od nedelje do subote)

Ukoliko bismo, recimo, želeli da pokrenemo skriptu *posalji_mejl.sh* svakog petka u 17:58,

cron komanda bi izgledala ovako:

```
58 17 * * 5 ./posalji_mejl.sh
```

U datom primeru, 58 i 17 predstavljaju 17 sati i 58 minuta, znak * na mestima za dan u mesecu i mesec označava sve moguće vrednosti ova dva polja, dok broj 5 na poslednjem mestu označava petak.

U okviru svoje sintakse, *cron* sadrži nekoliko rezervisanih karaktera koji predstavljaju određene operacije. Neke od tih operacija su:

- Zvezdica (*) – označava sve moguće vrednosti polja. Ukoliko bismo, na primer, želeli da se neki posao pokreće svaki minut, stavićemo vrednost * u okviru ovog polja.
- Zarez (,) – pomoću ovog operatora možemo navesti nekoliko vrednosti za jedno polje. Na primer, ukoliko kao vrednost polja za dan u nedelji stavimo 1,5 komanda će se izvršavati svakog ponedeljka i petka.
- Crtica (-) – operator – pomaže nam da definišemo opseg vrednosti. Ukoliko bismo za polje mesec stavili vrednost 6-9 komanda bi se izvršavala od juna do septembra.
- Kosa crta (/) – ovaj operator koristimo da bismo podelili određeno polje. Na primer, ukoliko u polje za sate unesemo */12 komanda će se izvršavati svakih 12 sati.

5. Korisno

5.1. Curl

Curl je komanda i biblioteka za prenos podataka koristeći *URL* sintaksu. Podržava mnoštvo protokola, kao što su *HTTP*, *HTTPS*, *FTP*, *FTPS*, *TELNET* i drugi. Komanda se najčešće koristi za preuzimanje fajlova sa interneta i testiranje *API*-ja (engl. *Application Programming Interface*).

Osnovni oblik komande je:

```
curl [url]
```

Tako, na primer, komandom `curl https://www.gnu.org/gnu/gnu.html` dobijamo odgovor servera u vidu *HTML* dokumenta stranice.

```
student@aperture: ~
student@aperture:~$ curl https://www.gnu.org/gnu/gnu.html
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
<head>
<meta http-equiv="content-type" content="text/html; charset=utf-8" />
<link rel="author" href="mailto:webmasters@gnu.org" />
<link rel="icon" type="image/png" href="/graphics/gnu-head-mini.png" />
<meta name="ICBM" content="42.355469,-71.058627" />
<link rel="stylesheet" type="text/css" href="/mini.css" media="handheld" />
<link rel="stylesheet" type="text/css" href="/layout.min.css" media="screen" />
<link rel="stylesheet" type="text/css" href="/print.min.css" media="print" />



<!-- Parent-Version: 1.96-->
<!-- This page is derived from /server/standards/boilerplate.html -->
<title>About the GNU Operating System
- GNU project - Free Software Foundation</title>
```

Slika 23 - Preuzimanje sadržaja stranice komandom curl

Pošto podržava *SMTP* protokol, pomoću komande *curl* moguće je slati mejlove.

```
curl --ssl-reqd \
  --url 'smtps://smtp.gmail.com:465' \
  --user 'posiljalac@gmail.com:lozinka_posiljaoca' \
  --mail-from 'posiljalac@gmail.com' \
  --mail-rcpt 'primalac@gmail.com' \
  --upload-file sadrzaj.txt
```

Slika 24 - Slanje mejla komandom curl

Komandi prosleđujemo nekoliko opcija. Opcijom `--ssl-reqd` navodimo da se koristi *SSL* protokol za uspostavljanje sigurne veze. Opciji `--url` dodeljujemo *URL* do *SMTP gmail* servera.

Opcije `--mail-from` i `--mail-rcpt` označavaju korisnika koji šalje i korisnika koji prima mejl, dok opciji `--upload-file` prosleđujemo sadržaj mejla koji se nalazi u fajlu tekstualnom fajlu.

5.2. Izlistavanje i manipulisanje procesima

Za početak, treba objasniti razliku između programa i procesa. Program je pasivan entitet koji se sastoji od niza instrukcija koje treba da se izvrše, dok je proces aktivan entitet koji predstavlja program **u izvršavanju**. Iz ovoga možemo zaključiti da program ima duži životni

vek, koji traje sve dok se nalazi u memoriji računara, dok proces ima organičen životni vek, jer se prekida nakon izvršenja svog zadatka. Sada ćemo više govoriti o procesima.

Za izlistavanje procesa koji su trenutno pokrenuti koristimo komandu **ps**.

```
student@aperture:~$ ps
    PID TTY          TIME CMD
   4179 pts/0    00:00:00 bash
   5488 pts/0    00:00:00 bash
   7126 pts/0    00:00:00 ps
```

Slika 25 - Komanda ps

Kao što možemo videti na slici, komandom *ps* dobijamo neke infromacije o pocesima kao što su:

- **PID** (*Process ID*) - predstavlja identifikator samog procesa
- **TTY** (*Teletype*) - terminal u kom je proces pokrenut
- **STAT -** status procesa, odnosno, šta proces radi i gde se nalazi njegova memorija. (npr. oznaka S se koristi da označi da je proces uspavan (engl. *sleeping*), dok oznaka R govori da je proces u izvršavanju (engl. *running*)
- **TIME -** količina procesorskog vremena koje je proces koristio
- **CMD** (*Command*) - naziv procesa, odnosno, komande kojom je pokrenut

Komanda *ps* ima razne opcije, pomenućemo neke od njih:

Tabela 1 - Opcije komande ps

ps x	Prikazuje sve procese koji su pokrenuti, a čiji je vlasnik trenutni korisnik
ps ax	Prikazuje sve procese na sistemu, ne samo one čiji je vlanik trenutni korisnik.
ps u	Prikazuje detaljnije informacije o samom procesu.
ps w	Prikazuje puna imena komandi.

Ove opcije se mogu kombinovati, pa je tako moguće pozvati komandu *ps aux* koja će izlistati sve procese na sistemu uz detaljne informacije.

```
student@aperture: ~
student@aperture:~$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0 170884 12820 ?        Ss   00:27   0:07 /sbin/init sp
root           2  0.0  0.0      0     0 ?        S    00:27   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        I<   00:27   0:00 [rcu_gp]
root           4  0.0  0.0      0     0 ?        I<   00:27   0:00 [rcu_par_gp]
root           5  0.0  0.0      0     0 ?        I<   00:27   0:00 [netns]
root           7  0.0  0.0      0     0 ?        I<   00:27   0:00 [kworker/0:0H
root           9  0.0  0.0      0     0 ?        I<   00:27   0:00 [kworker/0:1H
root          10  0.0  0.0      0     0 ?        I<   00:27   0:00 [mm_percpu_wq
root          11  0.0  0.0      0     0 ?        S    00:27   0:00 [rcu_tasks_ru
```

Slika 26 - Opcije aux komande ps

Ako želimo da prekinemo izvršavanje nekog procesa, poslaćemo mu signal sa ***kill*** komandom. Signal je poruka koju kernel šalje nekom procesu. Da bismo „ubili“ neki proces, komandi *kill* ćemo prosediti *PID* tog procesa. Postoji više tipova signala koji se prosleđuju komandi *kill* kao opcija:

- **TERM** (*terminate*) - podrazumevana opcija koja označava prekidanje procesa
- **STOP** - signal za stopiranje procesa, proces je još u memoriji i čeka da nastavi svoje izvršavanje
- **CONT** (*continue*) - nastavlja izvršavanje stopiranog procesa

```
student@aperture:~$ ps aux | grep gedit
student    49974  1.8  0.4 410996 57400 pts/1    Sl+  19:37   0:00 gedit notes.t
xt
student    50021  0.0  0.0  11912   724 pts/2    S+   19:38   0:00 grep --color=
auto gedit
student@aperture:~$ kill 49974
```

Slika 27 - "Ubijanje" procesa

Na slici iznad smo izlaz naredbe *ps aux* pomoću *pipe*-a prosledili naredbi *grep* koja filtrira izlaz naredbe *ps aux* tako da nam, kao rezultat, prikaže samo procese programa *gedit*. Prvi od njih ima *PID* 49974. Ovaj proces možemo prekinuti naredbom `kill 49974`.

Proces pokrenut u terminalu možemo da prekinemo i pomoću prečice **CTRL+C**, što će prekinuti proces **INT** (*interrupt*) signalom.

Objasnili smo način rada sa komandom *ps*, ali nam ona ne govori kako se procesi menjaju kroz vreme, i ono što je još bitnije, ne objašnjava nam koji su problematični procesi (oni koji koriste dosta procesorskog vremena ili zauzimaju previše memorije) i kako da ih pronađemo.

Program `top` nam govori dosta više o ovim stvarima, zato što prikazuje prvo one procese koji zauzimaju najviše memorije ili troše najviše procesorskog vremena. Ovaj program, takođe,

ažurira informacije svake sekunde. Program **`htop`** je noviji i pruža nam više funkcionalnosti od programa **`top`**, kao što su naglašavanje stvari bojama ili razne operacije klikom miša.

```
  0[||||||||||||||                                28.7%]   Tasks: 156, 635 thr; 2 running
  1[||||||||||||||                                26.9%]   Load average: 2.15 1.37 0.74
  2[||||||||||||||                                26.5%]   Uptime: 01:31:36
  3[||||||||||||||||||                            36.1%]
Mem[||||||||||||||||||||||||||||||||||||||||| 2.90G/7.66G]
Swp[                                           0K/2.00G]

  PID USER      PRI  NI  VIRT   RES   SHR S CPU%▽MEM%   TIME+  Command
 7318 pavle      20   0 1129G  147M 94448 S 67.3  1.9  0:29.13 /opt/google/chrome/chrome --type=renderer
 4654 pavle      20   0 33.5G  180M  116M S 31.3  2.3  2:38.37 /opt/google/chrome/chrome --type=gpu-proce
 6502 pavle      20   0 1129G  175M  107M S  9.5  2.2  0:39.69 /opt/google/chrome/chrome --type=renderer
 1951 pavle      20   0 1637M  119M 75516 R  8.2  1.5  1:35.43 /usr/lib/xorg/Xorg vt2 -displayfd 3 -auth
 4766 pavle      20   0 33.5G  180M  116M S  7.5  2.3  0:39.79 /opt/google/chrome/chrome --type=gpu-proce
 6512 pavle      20   0 1129G  175M  107M S  6.1  2.2  0:15.53 /opt/google/chrome/chrome --type=renderer
 7327 pavle      20   0 1129G  147M 94448 S  4.1  1.9  0:01.54 /opt/google/chrome/chrome --type=renderer
 4765 pavle      20   0 33.5G  180M  116M S  3.4  2.3  0:18.54 /opt/google/chrome/chrome --type=gpu-proce
 3517 pavle      20   0 1192M 82744 52592 S  2.7  1.0  3:05.42 /snap/teams/8/usr/share/teams/teams --type
```

Slika 28 - Rezultat htop programa

Kao što se sa slike može primetiti, program ***htop*** nam prikazuje i neka dodatna polja koja ćemo dodatno razjasniti:

- **USER -** predstavlja korisnika koji je vlasnik trenutnog procesa
- **PRI** (***Priority***) - predstavlja kernelov trenutni prioritet izvršavanja procesa
- **NI** (***Nice value***) - ova vrednost daje nagoveštaj kernelu koji proces treba da se sledeći izvrši. Ovom vrednošću može upravljati korisnik i na taj način uticati na odluku rasporeda izvršavanja procesa. Podrazumevana vrednost je 0 i ona ne utiče na odluku kernela, dok manja vrednost označava veći prioritet
- **VIRT -** predstavlja ukupan utrošak virtuelne memorije za izvršavanje procesa
- **RES -** predstavlja ukupan utrošak fizičke memorije za izvršavanje procesa
- **SHR -** predstavlja ukupan utrošak deljene memorije za izvršavanje procesa
- **CPU% -** predstavlja zauzetost procesora u procentima u toku izvršavanja procesa
- **MEM% -** predstavlja zauzetost memorije u procentima u toku izvršavanja procesa

Naglasićemo da su polja *S* i *Command* ekvivalenti poljima *STAT*, odnosno *COMMAND*, koje smo prethodno objasnili kod naredbe **`ps`**.

5.3. &

Kada pokrenemo neki proces iz terminala, u istom terminalu ne možemo kucati i pokretati druge naredbe dok se taj proces ne završi. Ovo zapravo ne mora biti slučaj, jer možemo da odvojimo proces od *shell*-a i da ga stavimo da radi u pozadini dok je naš *shell prompt* aktivan za izvršavanje drugih komandi. To postižemo stavljanjem znaka ampersand (&) na kraju

komande. Na primer, želimo da otvorimo *gedit* tekstualni editor, ali znamo da nećemo moći da koristimo *shell prompt* dokle god nam je tekstualni editor pokrenut. Zato ćemo na kraj komande dodati znak ampersand (&) koji će taj editor otvoriti u pozadini a vratiće nam njegov *PID*, kao što možemo primetiti na sledećoj slici. Iako je program *gedit* pokrenut, mi možemo izvršavati druge naredbe u terminalu.

Slika 29 - Pokretanje naredbe sa znakom &